

Desafio Técnico — API Laravel + React (Veículos)

Nova versão adaptada do desafio em MVC para uma arquitetura API + Front-end separado. O back-end deve ser construído em **Laravel 12** expondo **APIs REST** e o front end em **React**, consumindo essas APIs.

Objetivo

Entregar uma pequena aplicação **SaaS (multiusuário simples)** com:

- **Back-end (Laravel 12):** API REST autenticada para CRUD completo de **Veículos** e **Imagens de veículos**.
- **Front-end (React):** SPA que consome a API para **cadastrar, listar, pesquisar, visualizar, editar e excluir veículos**, fazer **upload de múltiplas imagens**, escolher **imagem de capa**, e exibir **auditoria simples**.

O foco é aplicar boas práticas de arquitetura, segurança, validação, documentação de API e UX no consumo das APIs.

Arquitetura Geral

- Aplicação dividida em **dois repositórios**:
 - **autoconf-vehicles-api** (Laravel 12).
 - **autoconf-vehicles-web** (React).
 - Comunicação via **HTTP/JSON**. Formatos binários apenas para upload de imagens (**multipart/form-data**).
 - **CORS** habilitado no back-end para permitir o domínio do front-end.
-

Back-end — Laravel 12 (API)

1) Stack e Diretrizes

- **Laravel 12** puro (sem Livewire/Inertia).
- Eloquent, Migrations, Seeders, **Policies** e **Form Requests**.
- **Armazenamento de imagens:** `storage/app/public` (com `php artisan storage:link`).
- **Padrões:** PSR-12, controllers enxutos; use Services/Actions quando fizer sentido.

2) Autenticação & Autorização

- **Sanctum** para autenticação baseada em sessão (SPA) ou **Personal Access Tokens** (PAT) — descreva a escolha.
- Usuário com papel opcional **admin** (coluna booleana ou enum). Autorização com **Policies** (**VehiclePolicy**). Apenas o **dono do veículo** (ou **admin**) pode editar/excluir.

3) Modelo de Domínio

Entidade **Vehicle**

- **placa** (string, única, validação BR — ex: ABC1D23)
- **chassi** (string, única, 17 chars)
- **marca** (string)
- **modelo** (string)
- **versao** (string)
- **valor_venda** (decimal(15,2))
- **cor** (string)
- **km** (integer >= 0)
- **cambio** (enum: **manual**, **automatico**)
- **combustivel** (enum: **gasolina**, **alcool**, **flex**, **diesel**, **hibrido**, **eletrico**)
- **user_id** (dono/criador) — multiusuário simples
- **timestamps**

Entidade **VehicleImage**

- **vehicle_id** (FK)
- **path** (string relativo ao storage)
- **is_cover** (bool; **apenas uma capa por veículo**)
- **timestamps**

Regras adicionais

- **placa** e **chassi** únicos por aplicação.
- Exatamente **uma** imagem com **is_cover** = **true** por veículo (garantir em DB ou lógica).
- Remover **arquivo físico** ao excluir imagem/veículo.

4) Endpoints (sugestão)

Base URL: **/api**

Auth

- `POST /auth/register` — cria usuário (`name`, `email`, `password`).
- `POST /auth/login` — autentica e retorna token/sessão.
- `POST /auth/logout` — invalida token/sessão vigente.
- `GET /auth/me` — retorna usuário autenticado.

Veículos

- `GET /vehicles` — lista com **paginação**, **filtro** e **ordenação**.
 - Query params:
 - `q` (busca global por `placa`, `marca`, `modelo`)
 - `marca`, `modelo`, `placa`
 - `sort` (ex.: `sort=km,-valor_venda`)
 - `page`, `per_page`
- `POST /vehicles` — cria veículo.
- `GET /vehicles/{id}` — detalhes do veículo (inclui imagens e metadados de auditoria).
- `PUT/PATCH /vehicles/{id}` — atualiza veículo.
- `DELETE /vehicles/{id}` — exclui veículo e suas imagens (em transação), removendo arquivos.

Imagens

- `POST /vehicles/{vehicleId}/images` — **upload múltiplo** (`files[]`), `multipart/form-data`. Retorna objetos imagem criados.
- `PATCH /vehicles/{vehicleId}/images/{imageId}/cover` — define capa (garanta singularidade).
- `DELETE /vehicles/{vehicleId}/images/{imageId}` — remove imagem (DB + arquivo físico).

Auditoria simples

- Popular `created_by` / `updated_by` (ou usar `user_id` + events) e expor no `GET /vehicles/{id}`.

5) Validações (Form Requests)

- `placa`: regex Mercosul (ex.: `/^[A-Z]{3}[0-9][A-Z0-9][0-9]{2}$/i`).
- `chassi`: tamanho 17, alfanumérico (sem I,O,Q se desejar).
- `valor_venda`: min 0.01.
- `km`: min 0.
- `cambio` e `combustivel`: `in:....`
- Imagens: `image`, tipos comuns, máx. **2MB** cada.

6) Documentação e Observabilidade

- Documentar endpoints via **OpenAPI (Swagger)** (ex.: [15-swagger](#)) ou **scribe**.
- Retornar erros padronizados (RFC 7807 opcional), com mensagens claras de validação.
- Logging adequado e tratamento de exceções.

7) Segurança

- **CSRF** (se usar sessão para SPA) ou **Bearer token (PAT)** para chamadas.
- **CORS** configurado apenas para os domínios do front-end.
- Proteção contra **Mass Assignment**, rate limiting em rotas sensíveis ([Throttle](#)).

8) Qualidade

- Testes de **Feature/Unit** com **Pest/PHPUnit** para regras críticas (auth, policy, validações, upload, capa única).
- Seeds para dados de exemplo (10 veículos com imagens placeholder).
- **README** claro (setup, [.env](#), storage link, seed, como gerar token e usar a doc da API).

Nota: Como o desafio é de curta duração, caso não seja possível implementar todos os testes, é suficiente entregar **exemplos em apenas um dos casos** (por exemplo: validações de Vehicle ou upload de imagens). Quanto mais testes forem entregues, melhor será para a validação do conhecimento.

Front-end — React (SPA)

1) Stack e Diretrizes

- React com Vite — livre escolha de composição (Hooks/Function Components).
- Gerenciador de estado/servidor: Redux, Recoil, Zustand, ou React Query/TanStack Query (React Query é um plus).
- Rotas com React Router.
- TypeScript é um plus.

2) Autenticação

- Tela de Login e Registro consumindo os endpoints [/auth/*](#).
- Armazenar o token de forma segura (ex.: cookie HttpOnly se usar sessão/Sanctum; estado em memória + refresh quando usar PAT).
- Guards no Router para proteger rotas autenticadas.

3) Telas Obrigatórias

- **Lista de Veículos:** tabela ou cards com paginação, busca (`q`), filtros (`marca`, `modelo`, `placa`) e ordenação (`km`, `valor_venda`) usando os query params da API.
- **Cadastro/Edição de Veículo:** formulário com validações client-side alinhadas com as validações da API e máscaras básicas opcionais.
- **Detalhe do Veículo:** exibição dos campos, galeria de imagens, ação para definir capa e metadados de auditoria.
- **Upload de Imagens:** envio de múltiplos arquivos via `multipart/form-data` para `POST /vehicles/{id}/images`, com ações para definir capa e excluir.

4) UX

- **Feedback visual** para loading, erro e sucesso em todas as operações de API.
- **Diálogos de confirmação** para ações destrutivas (excluir veículo/imagem).
- Exibir mensagens de validação retornadas pela API diretamente nos formulários.
- Layout responsivo.

5) Qualidade

- Organização de pastas por feature (ex.: `features/vehicles`, `features/auth`).
- Criação de hooks customizados para chamadas à API (`useVehicles`, `useVehicleImages`, etc.).
- **README** do front-end com passos para rodar e para configurar a `VITE_API_BASE_URL`.

6) Entregáveis do front-end

Repositório público `autoconf-vehicles-web` (React):

- Código-fonte React.
 - `README.md` com:
 - `npm i` ou `pnpm i`
 - Configuração de `VITE_API_BASE_URL`
 - Como rodar (`npm run dev`)
-

Requisitos Não-Funcionais

- **Paginação** consistente (`page`, `per_page`) e metadados no JSON (`total`, `last_page`, etc.).
- **Ordenação** por múltiplos campos (`sort=km,-valor_venda`).
- **Filtros** combináveis.

- **CORS** e **rate limit** apropriados.
- **Remoção física** de arquivos ao excluir entidades relacionadas.

Entregáveis

- **Dois repositórios públicos (GitHub):**
 - a. **Back-end (autoconf-vehicles-api):**
 - Código-fonte.
 - **README.md** com:
 - `cp .env.example .env` e configurar DB
 - `composer install`
 - `php artisan key:generate`
 - `php artisan migrate --seed`
 - `php artisan storage:link`
 - Como rodar (`php artisan serve`)
 - Como autenticar (Sanctum PAT ou sessão) e **Swagger URL** (se houver)
 - Usuário seed (ex.: `admin@example.com` / senha `password`)
 - b. **Front-end (autoconf-vehicles-web):**
 - Código-fonte.
 - **README.md** com:
 - `npm i / pnpm i`
 - Configurar `VITE_API_BASE_URL`
 - Como rodar (`npm run dev`)
- **Importante:** todo o projeto deve ser preparado para **execução em ambiente local**. Não é necessário configurar infraestrutura externa.
- **Apresentação final:** ao concluir o desafio, deverá ser realizada uma **apresentação online**, onde o candidato demonstrará o funcionamento do sistema, explicará decisões técnicas e mostrará o código.

Formato do Desafio (curta duração)

- Este é um **desafio de curta duração**. Caso não seja possível finalizar **todos** os itens, **priorize** o núcleo funcional: 1) Autenticação (login/registro), 2) CRUD de Veículos, 3) Upload/gestão de imagens com definição de capa, 4) Listagem com paginação/filtros/ordenação.
- **Testes e exemplos:** se o tempo estiver curto, entregue **pelo menos um** dos seguintes como exemplo prático (quanto mais, melhor para validar conhecimento):
 - **Testes de Feature/Unit** (Pest/PHPUnit) cobrindo um fluxo crítico (ex.: criação de veículo + validação de placa, ou upload e definição de capa).
 - **Seeders/Factories** com dados de exemplo para rodar localmente.
 - **Coleção Postman/Insomnia** com requests principais (auth, vehicles, images) e exemplos de payloads.
 - **README** com exemplos de requisição/resposta (cURL) e passos para reproduzir localmente.

Apresentação prática (encerramento)

- Ao finalizar, haverá um **teste prático com apresentação online**, onde você deverá:
 - Demonstrar a aplicação **rodando localmente** (API + front-end).
 - Explicar **arquitetura e decisões técnicas** (auth, policies, validações, storage local, CORS, paginação/filtros, tratamento de uploads, erro/logs).
 - Mostrar **documentação** (Swagger/Scribe/README) e **coleção de requests** (se houver).
 - Evidenciar **testes** (execução e cobertura do(s) caso(s) exemplo(s)).

Critérios de Avaliação

1. **Design da API** (clareza, consistência, status codes, validações).
2. **Segurança** (auth, policies, CORS, rate limit, mass assignment).
3. **Qualidade de código** (organização, padrões, testes, logs/erros).
4. **UX do front-end** (estado, feedbacks, acessibilidade básica, responsividade).
5. **Documentação** (READMEs, instruções de rodar, exemplos de requisição/resposta, Swagger).
6. **Boas práticas** (commits descritivos, separação de responsabilidades, tratativa de upload e capa única).

Extras (Bônus)

- **OpenAPI** completa e página Swagger.
- **E2E** com Cypress/Playwright (fluxos principais).
- **CI** (GitHub Actions) com testes e lints.

Observações Finais

- Você é livre para escolher bibliotecas no React (UI, formulários, upload, query), desde que **documente a decisão**.
- O escopo deve ser entregue em **ambiente local**, com repositórios públicos no GitHub.
- Lembre-se: **quanto mais testes, melhor**, mas exemplos em apenas um caso já serão aceitos pela limitação de tempo.